



Consensus in Distributed System



SOURAJIT BHATTACHARJEE · Follow

6 min read · Jan 31, 2023



583



4



Sahil: Hi Sourajit! Let's play something.

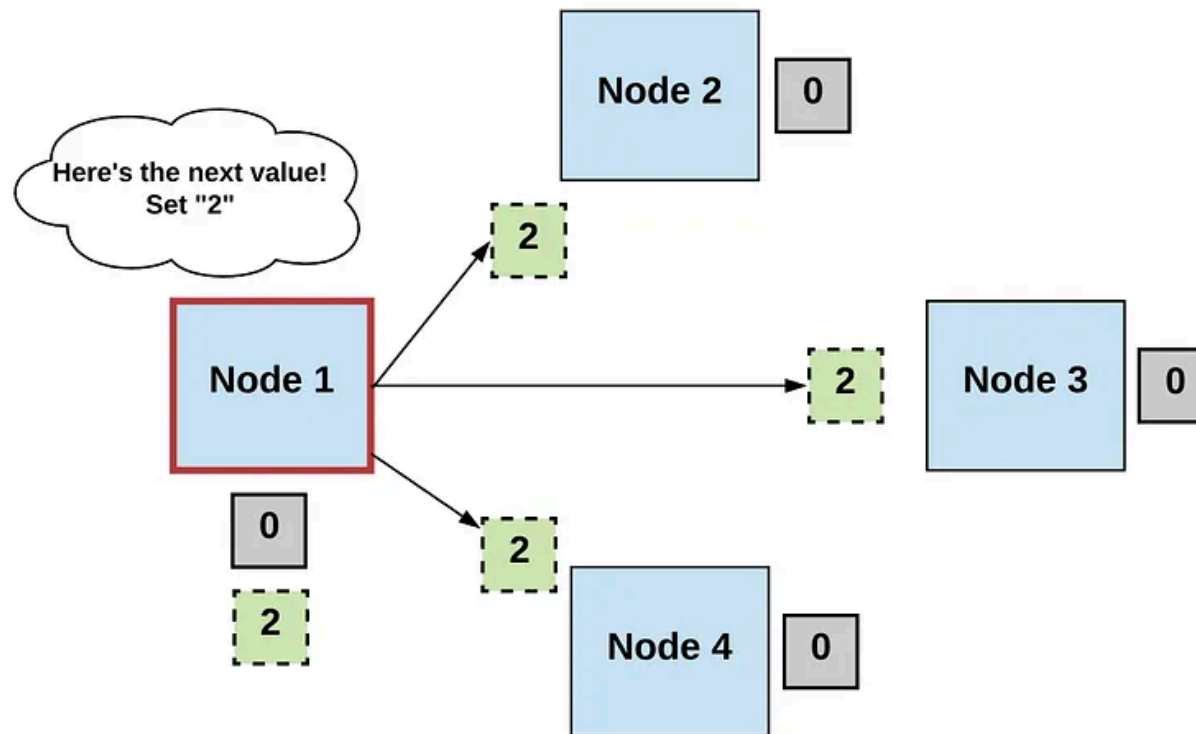
Sourajit: Let's play football.

Sahil: Yeah sure! We both love to play that.

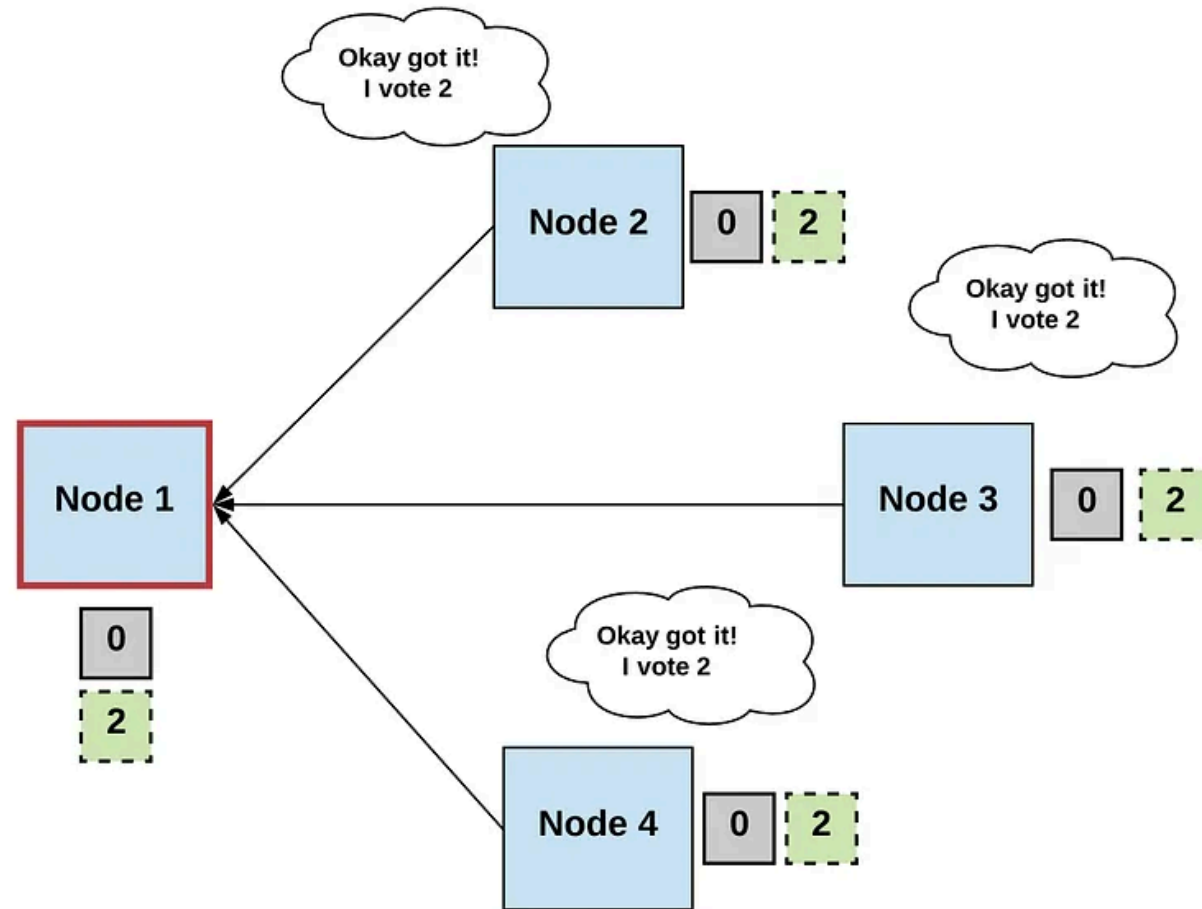
Both Sahil and Sourajit wanted to play a game and Sourajit suggested playing football. Sahil agreed with Sourajit's proposal. So, they agreed upon a common value suggested by one of them and took action on that. This agreement on common value is known as **consensus**.

What is Consensus in Distributed System?

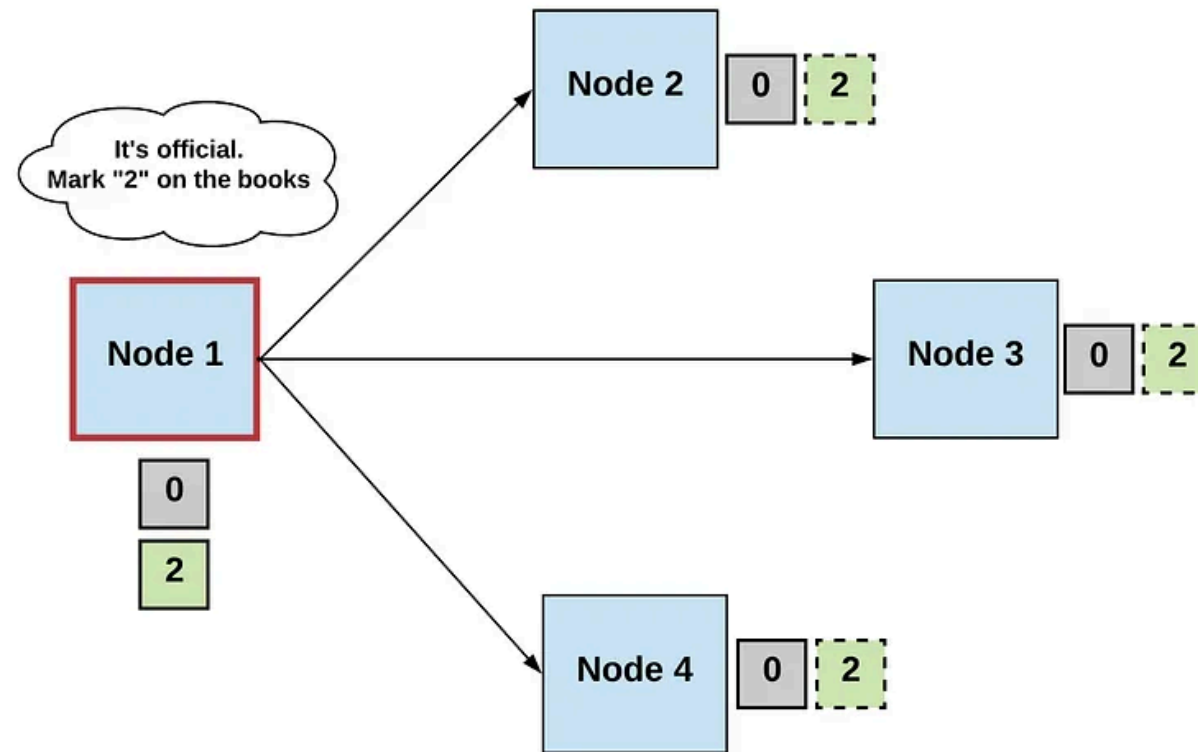
In a distributed system, multiple computers (known as **nodes**) are mutually connected with each other and collaborate with each other through message passing. Now, during computation, they need to agree upon a common value to coordinate among multiple processes. This phenomenon is known as **Distributed Consensus**.



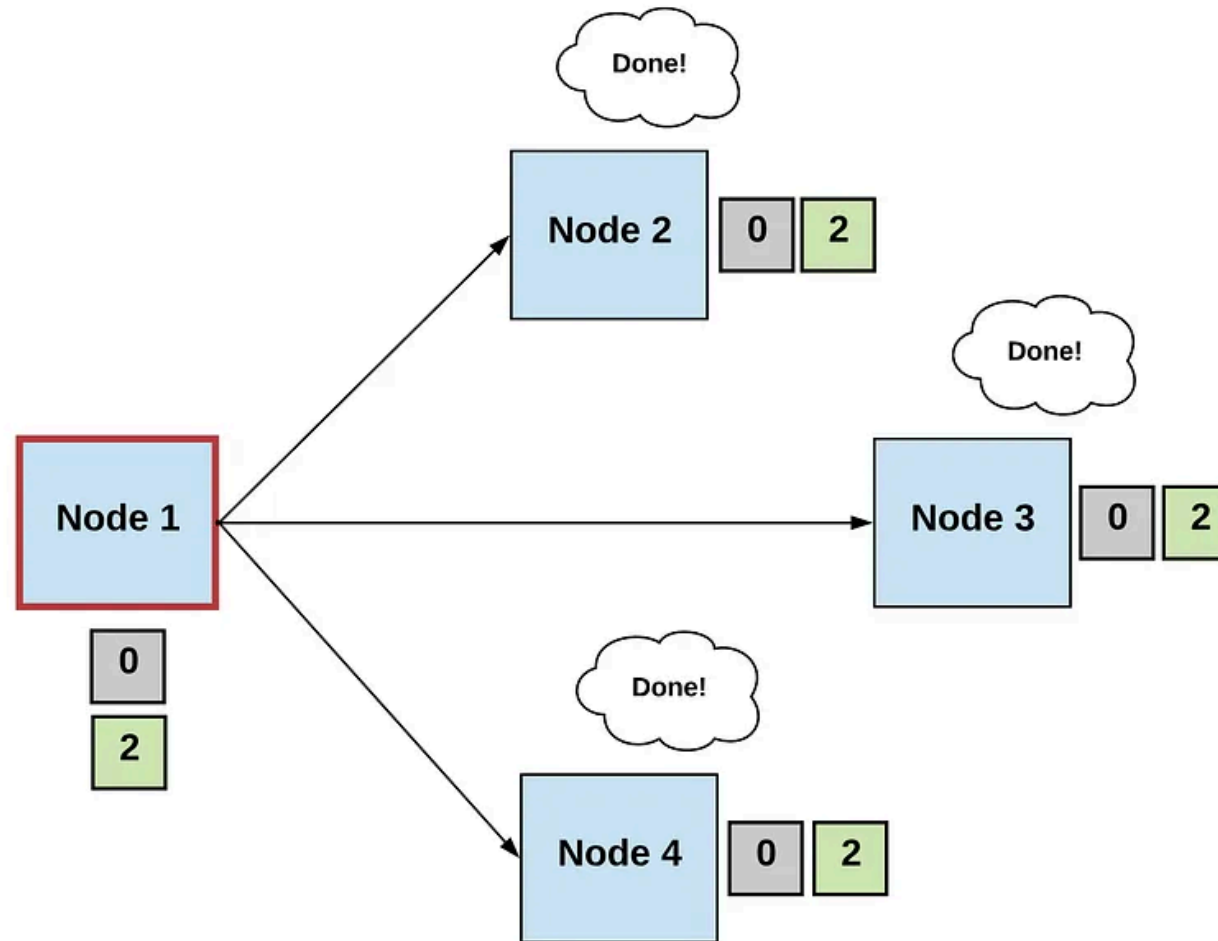
Source: <https://www.preethikasireddy.com/post/lets-take-a-crack-at-understanding-distributed-consensus>



Source: <https://www.preethikasireddy.com/post/lets-take-a-crack-at-understanding-distributed-consensus>



Source: <https://www.preethikasireddy.com/post/lets-take-a-crack-at-understanding-distributed-consensus>



Source: <https://www.preethikasireddy.com/post/lets-take-a-crack-at-understanding-distributed-consensus>

Why is Consensus required?

In a distributed system, it may happen that multiple nodes are processing large computations distributedly and they need to know the results of each

node to keep them updated about the whole system. In such a situation, the nodes need to agree upon a common value. This is where the requirement for consensus comes into the picture.

How to achieve Consensus in Distributed System

Let's consider a distributed system, where n nodes are connected with each other. one node sends a message to all other nodes saying "*Guys! I chose value v do you all agree?*".

Now, two scenarios can come up.

1. All agreed-upon values v
2. Some of them disagree

Case 1: All the nodes can carry out their task keeping v as the value in their mind.

Case 2: Suppose one of the nodes suggests "*I prefer value w* ". In this type of situation achieving consensus is complicated.

To achieve consensus all the nodes in distributed system should follow the same protocol when communicating. There are three basic conditions that need to be satisfied by the system for consensus algorithms.

1. **Agreement:** All non-faulty nodes should agree on the same value.
2. **Validity:** If a system has decided on a value v then that value should be suggested by one of the non-faulty nodes of the system and all other non-faulty nodes must decide that value v .
3. **Termination:** Every non-faulty node should agree upon some value. If one of the non-faulty nodes agrees upon value v , consensus can not be achieved.

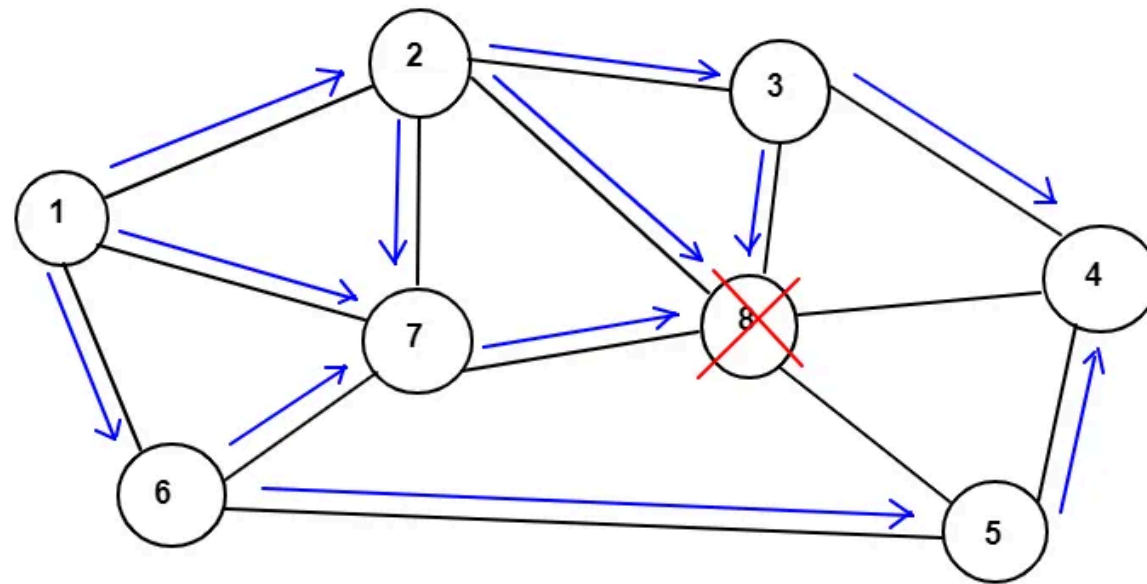
Non-faulty node means, node which is not crashed or attacked or malfunctioning.

Challenges in Distributed Consensus

A distributed system can face mainly two types of failure.

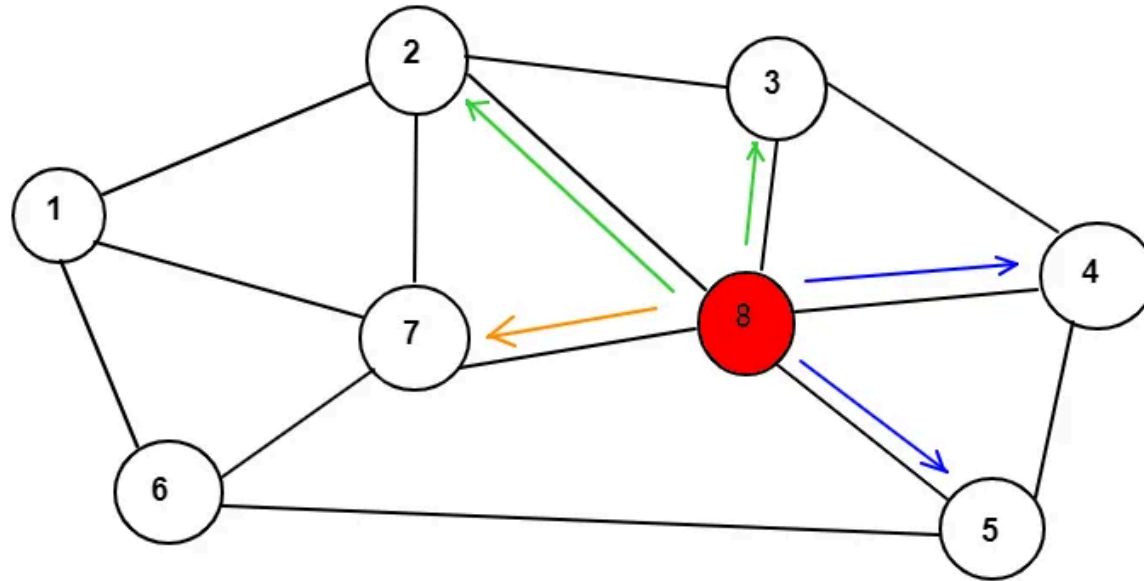
1. Crash failure
2. Byzantine failure

Crash failure occurs when a node is not responding to other nodes of the system due to some hardware or software or network fault. This is a very common issue in distributed systems and it can be handled easily by simply ignoring the node's existence.



Crash Failure

Byzantine failure is a situation where one or more node is not crashed but behaves abnormally and forward a different message to different peers, due to an internal or external attack on that node. Handling this kind of situation is complicated in the distributed system.



Byzantine Failure: faulty node sending different messages to different peers

A consensus algorithm, if it can handle Byzantine failure can handle any type of consensus problem in a distributed system.

Consensus Algorithms

Voting-based Consensus Algorithms

Some of the first implementations of consensus algorithms began to employ various voting-based techniques. These have enough fault tolerance and sufficient mathematical proof to assure security and stability. However,

because of their democratic character, these algorithms are extremely slow and inefficient, especially as the network becomes larger.

1. Practical Byzantine Fault Tolerance

Let's understand Practical Byzantine Fault Tolerance (pBFT) through an example.

Imagine that several divisions of the Byzantine army are camped outside an enemy city, each division commanded by its own general.

The generals can communicate with one another only by messenger. After observing the enemy, they must decide upon a common plan of action.

However, some of the generals may be traitors, trying to prevent the loyal generals from reaching an agreement. The generals must decide on when to attack the city, but they need a strong majority of their army to attack at the same time.

The generals must have an algorithm to guarantee that (a) all loyal generals decide upon the same plan of action, and (b) a small number of traitors cannot cause the loyal generals to adopt a bad plan.

The loyal generals will all do what the algorithm says they should, but the traitors may do anything they wish. The algorithm must guarantee condition (a) regardless of what the traitors do. The loyal generals should not only reach an agreement but should agree upon a reasonable plan.

Father of Distributed Systems *Leslie Lamport* proved that —

*If more than **two-thirds** of all nodes in a system are honest then consensus can be reached.*

This algorithm works on the above-mentioned principle. The distributed system is divided into three phases (*pre-prepare, prepare, commit*) and nodes are sequentially ordered with one node being the **Primary node (or leader node)** and others as the **Secondary node (or backup node)**. The objective is that all non-faulty nodes help in achieving a consensus regarding the state of the system using the majority rule.

pBFT consensus rounds are —

- The client sends a request to the primary node.
- The primary nodes broadcast the request to all secondary nodes.

- All the nodes perform the service that is requested and send it to the client as a reply.
- The request is served successfully when the client received a similar message from at least two-thirds of the total nodes.

The primary node is replaced in every consecutive round using the *view change protocol* if a predetermined amount of time passes without the leading node broadcasting a request to the backup (secondary) nodes.

2. Other Notable Algorithms

There are other voting-based consensus algorithms like —

- HotStuff
- Paxos
- Raft etc...

Proof-based Consensus Algorithms

With the development of blockchain technology and distributed ledgers, networks became considerably broader and permissionless. For these

circumstances, a proof-based consensus technique seemed preferable. In this case, a participant must show adequate proof of something in order to contribute to decision-making.

There are several Proof-based Consensus algorithms —

- Proof of Work (PoW).
- Proof of Stake (PoS). etc...

Application of Distributed Consensus

Consensus algorithms are used in many real-world applications in distributed or decentralized networks —

- ✓ Blockchain and cryptocurrencies
- ✓ Google page-rank
- ✓ Load balancing etc...

References

1. Pease, M., Shostak, R., and Lamport, L. "Reaching Agreement in the Presence of Faults." Journal of the ACM Volume 27 Issue 2 April 1980 pp 228–234 <https://doi.org/10.1145/322186.322188>
2. Fischer, M.J. (1983). The consensus problem in unreliable distributed systems (a brief survey). In: Karpinski, M. (eds) Foundations of Computation Theory. FCT 1983. Lecture Notes in Computer Science, vol 158. Springer, Berlin, Heidelberg. https://doi.org/10.1007/3-540-12689-9_99
3. Yang, C., Liu, M., Wang, K., Zhao, F., Jiang, X. (2020). Review on Variant Consensus Algorithms Based on PBFT. In: Sun, X., Wang, J., Bertino, E. (eds) Artificial Intelligence and Security. ICAIS 2020. Communications in Computer and Information Science, vol 1254. Springer, Singapore. https://doi.org/10.1007/978-981-15-8101-4_5
4. <https://www.baeldung.com/cs/consensus-algorithms-distributed-systems#1-practical-byzantine-fault-tolerance>
5. <https://www.techtarget.com/whatis/definition/consensus-algorithm>
6. <https://www.geeksforgeeks.org/practical-byzantine-fault-tolerancepbft/>

Written by:-

1. Sourajit Bhattacharjee, MTech CSE, IIT Kharagpur
2. Sahil Mahapatra, MTech CSE, IIT Kharagpur

Distributed Systems

Consensus

Byzantine Fault Tolerance



Written by **SOURAJIT BHATTACHARJEE**

32 Followers

Follow



More from SOURAJIT BHATTACHARJEE

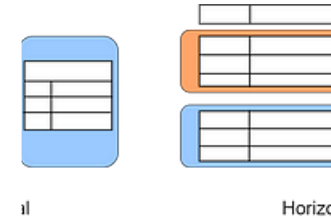


SOURAJIT BHATTACHARJEE

Sharding Technique in Database

What is sharding, why does it required, how it is done in database

Mar 8, 2023 🖱 51



See all from SOURAJIT BHATTACHARJEE

Recommended from Medium

JOURNAL, WPA

Mar. 2020 – May 2021

- ## Projects

- Visualized Google Takeout location data of location history using Google Maps API and Google Maps heatmap code with React
- Included local file system storage to reliably handle 5mb of location history data
- Implemented Express to include routing between pages and jQuery to parse Google Map and implement heatmap overlay

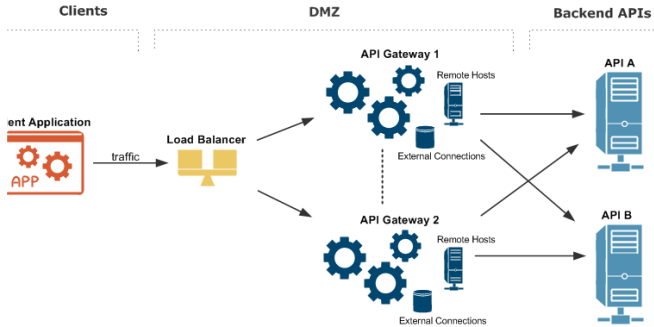
 ω^+

Developing a robust, scalable, and efficient system can be daunting. However,...

735 stories · 1313 saves



1713 stories · 1284 saves



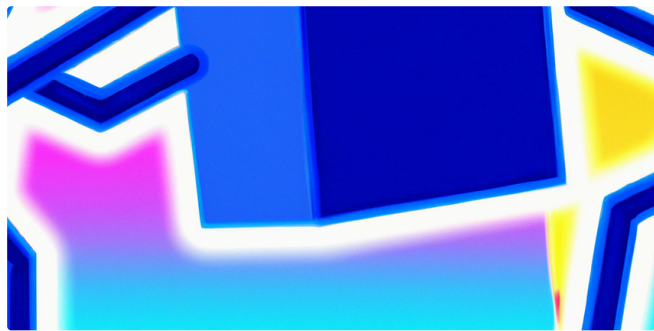
Sharath Kumar

API Gateway vs Load Balancer

API Gateway vs Load Balancer

Aug 3

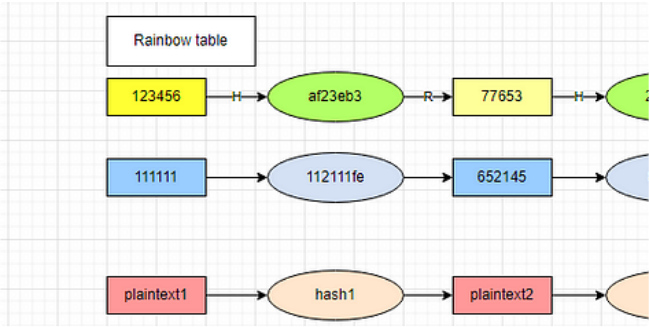
30



Vipul Kumar

API Throttling Vs Rate Limiting

API Throttling vs. Rate Limiting: Ensuring Stability in Your Application



LORY

A basic question in security Interview: How do you store...

Explained in 3 mins.



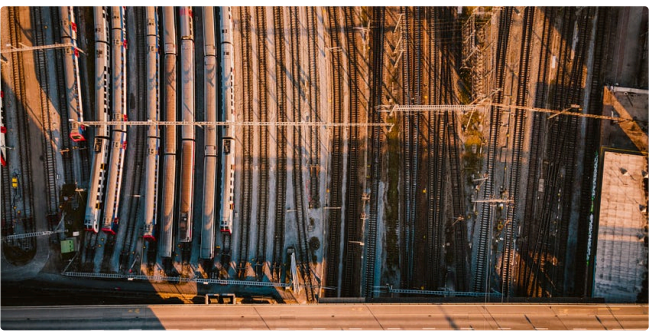
May 12



4.94K



62



Andrew Zuo

Async Await Is The Worst Thing To Happen To Programming

Jun 24



I recently saw this meme about async and await.



Jun 22



3.5K



209



See more recommendations

[Help](#) [Status](#) [About](#) [Careers](#) [Press](#) [Blog](#) [Privacy](#) [Terms](#) [Text to speech](#) [Teams](#)